I will provide you the cells of a jupyter notebook for you to fill in where it says TODO. I will run your code and continuously prompt you to fill everything in to get the right answer.

This si the context of the problem:

We will train a GNN to cluster people in the karate club in such that people who are more likely to associate with either the officer or Mr. Hi will be close together, while the distance beween the 2 classes will be far.

In the original paper titled "Semi-Supervised Classification with Graph Convolutional Networks" that can be found here https://arxiv.org/pdf/1609.02907.pdf, the authors framed this as a node-level classification problem on a graph. We will pretend that we only know the affiliation labels for some of the nodes (which we'll call our training set) and we'll predict the affiliation labels for the rest of the nodes (our test set).

We will build a multi-layer Graph Convolutional Network (GCN) with the following layer-wise propagation rule:

$$H^{(l+1)} = σ(D^{-1/2}\tilde{A} D^{-1/2}H^{(l)}W^{(l)})
= σ(\tilde{A}^{SymNorm}H^{(l)}W^{(l)})$$

where $\tilde{A}$ is the adjacency matrix that we discussed in Homework 4, except that now we add the identity to include self loops at every node for numerical stability. D is the degree matrix as defined in the past, $H^{(l)}$ is the activation matrix of the l-th layer, and W is the weight matrix to be learned. σ is an activation function, in this case tanh.

Step 1:
Q. 1. Compute: $$\tilde{A}=A_{self-loop}= A + I$$
Where I is the identity matrix which allows us to include self loops of each nodes.

TODO
A_tild = ????

STep 2:
Q.2. Write a function that takes in $\tilde A$ as argument and returns the $\tilde A^{SymNorm}$ adjacency matrix. You may find the provided function `fill_diagonal` useful, as well as the inverse function `np.linalg.inv` and the matrix square root function `np.linalg.sqrtm`.

def get_adjacency_matrix(A_tild):
  # TODO
  return

Let's fill in the TODOs for the Graph Convolutional Network (GCN) implementation based on the provided context. I'll provide the solutions for each step, ensuring they align with the mathematical formulation given in the problem.

Step 1: Compute $\tilde{A} = A + I$

The goal is to compute $\tilde{A}$, which is the adjacency matrix A with self-loops added. This is done by adding the identity matrix I to the adjacency matrix A. The identity matrix I has ones on the diagonal and zeros elsewhere, effectively adding a self-loop to each node in the graph.

Assuming A is the adjacency matrix of the karate club graph (typically loaded from a library like NetworkX or provided as a NumPy array), the code to compute $\tilde{A}$ is straightforward.

```python
# Step 1
# TODO
# A_tild = ????
```

```
import numpy as np

# Assuming A is the adjacency matrix provided as a NumPy array
A_tild = A + np.eye(A.shape[0])
```

Explanation:

- `np.eye(A.shape[0])` creates an identity matrix I with the same number of rows (and columns) as A, which corresponds to the number of nodes in the graph.
- Adding A + np.eye(A.shape[0]) ` adds a 1 to the diagonal elements of \(A, introducing self-loops for each node.
- The result, `A_tild`, is the adjacency matrix with self-loops, as required by $\tilde{A} = A + I$.

Step 2: Compute $\tilde{A}^{SymNorm}$

The goal is to write a function that computes the symmetrically normalized adjacency matrix $\tilde{A}^{SymNorm}$, defined as:

$$\tilde{A}^{SymNorm} = D^{-1/2}\tilde{A}D^{-1/2}$$

Here, $\tilde{A}$ is the adjacency matrix with self-loops (from Step 1), and D is the degree matrix, a diagonal matrix where each diagonal element D_{ii} is the degree of node i (including the self-loop, so the degree is the sum of the row of $\tilde{A}$).

The function should:

1. Compute the degree matrix D from $\tilde{A}$.
2. Compute $D^{-1/2}$, the inverse square root of the degree matrix.
3. Compute the symmetrically normalized adjacency matrix using the formula.

Here's the implementation:

```
python

# Step 2
def get_adjacency_matrix(A_tild):
    # TODO
    # Compute the degree matrix D
    degrees = np.sum(A_tild, axis=1)  # Sum of each row gives the degree
```



```
    D = np.diag(degrees)  # Create diagonal matrix with degrees

    # Compute D^{-1/2}
    D_inv_sqrt = np.diag(1.0 / np.sqrt(degrees))  # Inverse square root o

    # Compute symmetrically normalized adjacency matrix: D^{-1/2} * A_til
    A_sym_norm = D_inv_sqrt @ A_tild @ D_inv_sqrt

    return A_sym_norm
```

Explanation:

- **Degree Matrix D**:
  - `np.sum(A_tild, axis=1)` computes the degree of each node by summing the rows of $\tilde{A}$. Since $\tilde{A}$ includes self-loops, the degree of node i is the sum of the i-th row, which includes a 1 for the self-loop.
  - `np.diag(degrees)` creates a diagonal matrix where the diagonal elements are the degrees.
- **Inverse Square Root $D^{-1/2}$**:
  - `np.sqrt(degrees)` computes the square root of the degrees.
  - `1.0 / np.sqrt(degrees)` computes the inverse square root of the degrees.
  - `np.diag(1.0 / np.sqrt(degrees))` creates a diagonal matrix with the inverse square roots on the diagonal, forming $D^{-1/2}$.
- **Symmetrically Normalized Adjacency Matrix**:
  - The formula $D^{-1/2}\tilde{A}D^{-1/2}$ is implemented using matrix multiplication (`@` in NumPy).
  - `D_inv_sqrt @ A_tild @ D_inv_sqrt` computes the symmetrically normalized adjacency matrix $\tilde{A}^{SymNorm}$.

Notes:

- The function assumes `A_tild` is a NumPy array.
- The implementation avoids using `np.linalg.inv` and `np.linalg.sqrtm` directly, as the degree matrix is diagonal, making the inverse square root computation simpler and more efficient by working directly with the diagonal elements.

- If the graph has nodes with degree 0 (unlikely in the karate club dataset with self-loops), you might need to handle division by zero, but since self-loops ensure each node has at least degree 1, this is not a concern here.

Next Steps

Please provide the next part of the Jupyter notebook or let me know if you need further clarification or adjustments to the provided solutions. If you run the code and encounter issues or need additional cells filled, share the details, and I'll assist with the next TODOs or debug any problems.

> Using the above code we have run the following to get A-hat:
> A_hat = get_adjacency_matrix(A_tild)
> A_hat
>
> STep 3:
> The other input to our GNN is the graph node matrix X which contains node features. For simplicity, we set X to be the identity matrix because we don't have any node features in this example. In a sense, this will map each node in the graph to a column of learnable parameters in the first layer, resulting in a fully learnable node embedings. In the question below, set the matrix X to be the identity.
> Q.3. Generate the feature input matrix X
> # TODO
> # X = ??

Step 3: Generate the Feature Input Matrix X

The goal is to create the node feature matrix X for the Graph Convolutional Network (GCN). As specified, since no node features are provided in this example, we set X to be the identity matrix. This means each node in the graph is assigned a unique one-hot encoded vector as its initial feature, effectively mapping each node to a column of learnable parameters in the first layer of the GCN.

The identity matrix I has ones on the diagonal and zeros elsewhere, and its size should match the number of nodes in the graph, which is the same as the number of rows (or columns) of the adjacency matrix A or $\tilde{A}$.

Here's the implementation:

python

```python
# Step 3
# TODO
# X = ???

import numpy as np

# Assuming A_tild is the adjacency matrix with self-loops from Step 1
X = np.eye(A_tild.shape[0])
```

Explanation:

- `A_tild.shape[0]` gives the number of nodes in the graph, as the adjacency matrix $\tilde{A}$ is a square matrix where the number of rows (or columns) equals the number of nodes.
- `np.eye(A_tild.shape[0])` creates an identity matrix of size $N \times N$, where N is the number of nodes. This matrix has ones on the diagonal and zeros elsewhere, ensuring each node is represented by a unique one-hot vector.
- The resulting X is an $N \times N$ matrix, where each row corresponds to a node's feature vector (a one-hot vector in this case).

Notes:

- The code assumes `A_tild` is available from Step 1 and is a NumPy array representing the adjacency matrix with self-loops.
- Using the identity matrix as the feature matrix is a common approach when no explicit node features are available, as it allows the GCN to learn node embeddings through the weight matrices in subsequent layers.

Next Steps

Please provide the next part of the Jupyter notebook or let me know if you need further clarification, additional cells to fill, or help debugging any issues that arise when running the code. If you've run the code and have results or errors, share them, and I'll assist with the next TODOs or any necessary adjustments.

We will first implement a single layer GNN. Using the equation provided in the paper that is mentioned above, implement a forward and backward pass for a simple GNN layer.

Note that for l=0, H is the input X and AH does the message passing as we have seen in the previous homework and discussion, which is in turn multiplied by a weight matrix W. A non linearity is therefore applied afterward.

In the backward pass, we will apply L2 regularization to the weight matrix W. The regularization term is defined as: $\lambda \sum_{i,j} W_{i,j}^2$ which has gradient: $\lambda 2W$. We will combine $\lambda 2$ in the weight decay parameter optimizer.weight_decay.

Q.4. Complete the #TODO to implement a forward pass that does just that in the class below. DO not change any of the code otherwise:

```
class GNN_Layer():
  """process a single GNN layer"""
  def __init__(self, input_dim, output_dim, name=''):
    self.name = name
    self.cache = {}
    self.input_dim = input_dim
    self.output_dim = output_dim
    self.W = get_xavier_init(self.output_dim, self.input_dim)
    self.activation = np.tanh

  def __repr__(self):
    dims = (self.input_dim, self.output_dim)
    if self.name:
      return f"GNN_Layer: W{'_' + self.name}{dims}"
    else:
      return f"GNN_Layer: W{'_'+ ''}{dims}"
```



```
  def forward_pass(self, A, X, W=None):
    """A here is the symmetricaly normalized adjacency matrix
    and X is the input to the layer. We cached some values
    to use in the backward pass."""

    self.cache['A'] = A # (batch_size, batch_size)
    # TODO
    # self.cache['X'] = ?? # (input_node_feature_dim, batch_size)

    # if W is None:
      # W = ??

    # H = ?? (hidden_dim, batch_size)
    # H = ??  (apply activation)
    # self.cache['H'] = ?? # (hidden_dim, batch_size)
    return # (batch_size, hidden_dim)

  def backward_pass(self, optimizer, need_update=True):
    # dtanh = ?? # (batch_size, output_dim)
    # optimizer.output contains the gradient from the next layer. It is multiplied by the batch size,
    # so you'll need to divide by optimizer.batch_size to get the average gradient.
    d = np.multiply(optimizer.output, dtanh) # (batch_size, output_dim)

    # self.grad = ?? (batch_size, input_dim)

    optimizer.output = self.grad

    # dW = ?? # (output_dim, input_node_feature_dim)
    # dW_weight_decay = ??

    # if need_update: # Use the gradient descent update rule on W. Remember to include weight decay.
      # self.W = ??
```

```
    return # (output_dim, input_node_feature_dim)
```

Let's implement the forward and backward passes for the `GNN_Layer` class based on the provided equation for a Graph Convolutional Network (GCN) layer:

$$H^{(l+1)} = \sigma(D^{-1/2}\tilde{A}D^{-1/2}H^{(l)}W^{(l)}) = \sigma(\tilde{A}^{SymNorm}H^{(l)}W^{(l)})$$

Here, $\tilde{A}^{SymNorm}$ is the symmetrically normalized adjacency matrix (denoted as `A` in the code), $H^{(l)}$ is the input activation matrix (denoted as `X` for the first layer), $W^{(l)}$ is the weight matrix, and σ is the activation function (tanh in this case). For the first layer ($l = 0$), the input $H^{(0)}$ is the feature matrix X, which we set as the identity matrix in the previous step.

The backward pass includes computing gradients with respect to the inputs and weights, incorporating L2 regularization on the weight matrix W. The regularization term is $\lambda \sum_{i,j} W_{i,j}^2$, with its gradient being $2\lambda W$. This is combined into the `optimizer.weight_decay` parameter.

Below is the implementation for the `forward_pass` and `backward_pass` methods, filling in the TODOs without altering the provided class structure.

gnn_layer.py
python

Edit in files • Show inline

```
text

### Explanation

#### Forward Pass
The forward pass implements the layer-wise propagation rule \( H^{(l+1)}

- **Cache Input**:
  - `self.cache['X'] = X`: Stores the input matrix \( X \) (shape: `batch

- **Weight Matrix**:
  - `if W is None: W = self.W`: Uses the layer's weight matrix `self.W` (
```



```
- **Message Passing and Transformation**:
  - `H = A @ X @ W.T`: Computes \(\tilde{A}^{SymNorm} H^{(l)} W^{(l)}\).
    - \( A \) is the symmetrically normalized adjacency matrix (shape: `b
    - \( X \) is the input activation matrix (shape: `batch_size, input_d
    - \( W \) is the weight matrix (shape: `output_dim, input_dim`), so \
    - First, \( A @ X \) performs message passing, aggregating features f
    - Then, \( (A @ X) @ W.T \) applies the linear transformation, result

- **Activation**:
  - `H = self.activation(H)`: Applies the tanh activation function elemen

- **Cache Output**:
  - `self.cache['H'] = H`: Stores the activated output (shape: `batch_siz

- **Return**:
  - Returns `H` (shape: `batch_size, output_dim`), the output of the GCN

#### Backward Pass
The backward pass computes the gradients for backpropagation, incorporati

- **Derivative of Tanh**:
  - `dtanh = 1 - np.square(self.cache['H'])`: Computes the derivative of
  - `np.square(self.cache['H'])` computes the element-wise square of the

- **Gradient of the Loss with Respect to Pre-Activation**:
  - `d = np.multiply(optimizer.output / optimizer.batch_size, dtanh)`: Co
    - `optimizer.output` contains the gradient from the next layer (shape
    - Dividing by `optimizer.batch_size` computes the average gradient.
    - Element-wise multiplication with `dtanh` applies the chain rule for

- **Gradient with Respect to Input**:
  - `self.grad = self.cache['A'] @ d @ self.W`: Computes the gradient wit
    - The forward pass is \( H = \tilde{A}^{SymNorm} X W^T \), so the gra
    - \( d \) (shape: `batch_size, output_dim`) is the gradient of the lo
    - \( self.W \) (shape: `output_dim, input_dim`).
    - \( self.cache['A'] \) is \(\tilde{A}^{SymNorm}\) (shape: `batch_siz
    - \( self.cache['A'] @ d \) (shape: `batch_size, output_dim`) aggrega
    - \( (self.cache['A'] @ d) @ self.W \) results in `self.grad` (shape:
  - `optimizer.output = self.grad`: Passes the gradient backward to the p

- **Gradient with Respect to Weights**:
  - `dW = d.T @ self.cache['A'] @ self.cache['X']`: Computes the gradient
    - The forward pass involves \( \tilde{A}^{SymNorm} X W^T \), so the g
      - \( d \) (shape: `batch_size, output_dim`) is the gradient with re
```

```
      - \( self.cache['A'] @ self.cache['X'] \) (shape: `batch_size, inpu
      - \( d.T \) (shape: `output_dim, batch_size`).
      - \( d.T @ (self.cache['A'] @ self.cache['X']) \) results in `dW` (

- **Weight Decay**:
  - `dW_weight_decay = dW + optimizer.weight_decay * self.W`: Incorporate
    - The regularization term is \(\lambda \sum_{i,j} W_{i,j}^2\), with g
    - The code specifies that \(2\lambda\) is included in `optimizer.weig
    - Adding this to `dW` gives the total gradient with respect to \( W \

- **Weight Update**:
  - `if need_update: self.W = self.W - optimizer.lr * dW_weight_decay`: A
    - `optimizer.lr` is the learning rate.
    - `dW_weight_decay` is the total gradient, including the data loss an
    - The update subtracts the gradient scaled by the learning rate from

- **Return**:
  - Returns `dW_weight_decay` (shape: `output_dim, input_dim`), the gradi

### Notes
- **Shapes**:
  - \( A \): `batch_size, batch_size` (number of nodes, e.g., 34 for the
  - \( X \): `batch_size, input_dim` (e.g., `34, 34` if \( X \) is the id
  - \( W \): `output_dim, input_dim` (e.g., `output_dim, 34` for the firs
  - \( H \): `batch_size, output_dim` (e.g., `34, output_dim`).
  - \( d \): `batch_size, output_dim`.
  - \( self.grad \): `batch_size, input_dim`.
  - \( dW \): `output_dim, input_dim`.
- **Assumptions**:
  - `get_xavier_init` is assumed to be a function that initializes the we
  - `optimizer` has attributes `output` (gradient from the next layer), `
  - All matrices are NumPy arrays, and matrix multiplications are perform
- **Tanh Derivative**:
  - The derivative of \(\tanh(z)\) is \(1 - \tanh^2(z)\), computed effici

### Next Steps
Please provide the next part of the Jupyter notebook or let me know if yo
```

We now instantiate everything:

```
gnn_layer = GNN_Layer(input_dim=num_nodes,
            output_dim=2,
            name='gnn_layer_1')

sm_layer = Softmax_Layer(input_dim=2,
            output_dim=num_classes,
            name='softmax_layer')

optim = Grad_Descent_Optimizer(learning_rate=0,
                    weight_decay=1.)

now fill this in:
# TODO
# gnn_layer_output = gnn_layer.forward_pass(A=??,
#                                 X=??)

# optim(y_pred=sm_layer.forward_pass(X=gnn_layer_output),
y_true=labels)
```

Let's fill in the TODOs for the instantiation and forward pass of the GNN layer and the subsequent optimization step. The context indicates that we are working with the karate club dataset, where `num_nodes` is the number of nodes (e.g., 34 for the karate club graph), `num_classes` is likely 2 (for the two affiliations: Officer and Mr. Hi), and we have the following components:

- `A_hat` : The symmetrically normalized adjacency matrix from Step 2 (denoted as $\tilde{A}^{SymNorm}$).
- `X` : The node feature matrix, set as the identity matrix from Step 3.
- `labels` : The ground-truth labels for the nodes (likely a vector or one-hot encoded matrix indicating affiliations).
- `gnn_layer` : A GNN layer with `input_dim=num_nodes` (e.g., 34) and `output_dim=2` , producing node embeddings.
- `sm_layer` : A softmax layer mapping the GNN output to class probabilities.
- `optim` : A gradient descent optimizer with a learning rate (set to 0, which may need adjustment) and weight decay.

The TODOs involve calling the forward pass of the `GNN_Layer` with the appropriate inputs and then passing the output through the softmax layer and optimizer. Here's the

implementation:

gnn_forward.py
python

Edit in files • Show inline

```
text
```

```
### Explanation

#### Forward Pass of GNN Layer
- **Inputs**:
  - `A=A_hat`: The symmetrically normalized adjacency matrix (\(\tilde{A}
  - `X=X`: The node feature matrix from Step 3, set as the identity matri
- **Call**:
  - `gnn_layer.forward_pass(A=A_hat, X=X)`: Calls the forward pass of the
    \[
    H = \tanh(\tilde{A}^{SymNorm} X W^T)
    \]
    where \( W \) is the layer's weight matrix (shape: `output_dim, input
  - The output `gnn_layer_output` has shape `num_nodes, 2`, representing
- **Notes**:
  - The `W` parameter in `forward_pass` is not specified (defaults to `No
  - The `batch_size` in the `GNN_Layer` context refers to `num_nodes`, as

#### Optimizer and Softmax Layer
- **Softmax Forward Pass**:
  - `sm_layer.forward_pass(X=gnn_layer_output)`: Passes the GNN layer's o
  - The softmax layer maps the 2-dimensional embeddings to class probabil
  - This produces `y_pred`, the predicted probabilities for each node bel
- **Optimizer Call**:
  - `optim(y_pred=sm_layer.forward_pass(X=gnn_layer_output), y_true=label
  - `labels` is assumed to be provided (e.g., a one-hot encoded matrix of
  - The optimizer likely computes the loss (e.g., cross-entropy between `
- **Notes**:
  - The learning rate is set to `0` in the optimizer, which would prevent
  - The `weight_decay=1.0` corresponds to \(2\lambda\) in the L2 regulari

### Assumptions
- `A_hat` is the output of `get_adjacency_matrix(A_tild)` from Step 2, av
- `X` is the identity matrix from Step 3 (`np.eye(A_tild.shape[0])`).
- `labels` is available and correctly formatted (e.g., shape `num_nodes,
```



```
- `Grad_Descent_Optimizer` and `Softmax_Layer` are defined elsewhere and
- `num_nodes` is the number of nodes in the graph (e.g., 34 for the karat

### Potential Issues to Watch For
- **Learning Rate**: A learning rate of `0` in `optim` will prevent weigh
- **Label Format**: Ensure `labels` matches the expected format for the o
- **Shapes**: Verify that:
  - `A_hat`: `num_nodes, num_nodes`.
  - `X`: `num_nodes, num_nodes`.
  - `gnn_layer_output`: `num_nodes, 2`.
  - `y_pred`: `num_nodes, num_classes`.
  - `labels`: Compatible with `y_pred` (e.g., `num_nodes, num_classes` or

### Next Steps
Please run the provided code in your Jupyter notebook and share:
- Any errors or unexpected outputs.
- The next part of the notebook or additional TODOs to fill.
- If you need clarification or adjustments (e.g., if the optimizer's lear
- If you need help implementing the backward pass or further training ste

I'll assist with the next steps or debug any issues based on your feedbac
```

Our implementation is so far correct:

Now fill in this skeleton code. Only fill in where the code tells you to:
Q. 6. Now, use the GNN and Softmax layers implemented above to set up our GNN Network.

```
class GNN():
  """This class leverages the GNN layer implemanted above by cascading them into a layer."""
  def __init__(self,
               input_dim,
               output_dim,
               hidden_dim,
               num_layers,):

    self.input_dim = input_dim
    self.output_dim = output_dim
    self.hidden_dim = hidden_dim
```

```
        self.num_layers = num_layers

        self.layers = []
#       first_gnn_layer = GNN_Layer(input_dim=??,
#                                   output_dim=??,
#                                   name='layer_0')

        self.layers.append(first_gnn_layer)

#       for layer in range(num_layers):
#           gnn_temp = GNN_Layer(input_dim=??,
#                                output_dim=??,
#                                name=f'layer_{layer}')
#           self.layers.append(gnn_temp)



#       last_gnn_layer = Softmax_Layer(input_dim=??,
#                                      output_dim=??,
#                                      name='sm_layer')
#       self.layers.append(last_gnn_layer)

    def __repr__(self):
        return '\n'.join([str(layer) for layer in self.layers])

    def embedding(self, A, X):
        H = X
        for layer in self.layers[:-1]:
            # H = ??
            pass
        # return

    def forward_pass(self, A, X):
        # H = ??
        out = self.layers[-1].forward_pass(H)
```



```
        pass
        # return
```

Let's fill in the TODOs for the `GNN` class, which constructs a multi-layer Graph Convolutional Network (GCN) by cascading `GNN_Layer` instances followed by a `Softmax_Layer`. The implementation will use the previously defined `GNN_Layer` and `Softmax_Layer` classes, along with the symmetrically normalized adjacency matrix `A_hat` and input feature matrix `X` (identity matrix) from earlier steps. The goal is to set up the network architecture and implement the forward pass to produce node embeddings and class probabilities for the karate club classification task.

Below is the completed code, filling in only the TODOs as instructed, wrapped in the required artifact tag.

gnn_network.py
python

Edit in files • Show inline

```
text

### Explanation of TODOs

#### Initialization (`__init__`)
- **First GNN Layer**:
  - `first_gnn_layer = GNN_Layer(input_dim=input_dim, output_dim=hidden_d
    - `input_dim`: The dimensionality of the input feature matrix \( X \)
    - `output_dim=hidden_dim`: The output dimensionality of the first GNN
    - `name='layer_0'`: As specified, names the first layer.
  - The layer is appended to `self.layers`.

- **Intermediate GNN Layers**:
  - `for layer in range(num_layers - 1)`: Loops to create `num_layers - 1
  - `gnn_temp = GNN_Layer(input_dim=hidden_dim, output_dim=hidden_dim, na
    - `input_dim=hidden_dim`: Each subsequent GNN layer takes the output
    - `output_dim=hidden_dim`: The output dimensionality remains `hidden_
    - `name=f'layer_{layer + 1}'`: Names layers sequentially (e.g., `laye
  - Each `gnn_temp` layer is appended to `self.layers`.
  - **Note**: The loop range is adjusted to `num_layers - 1` because the

- **Softmax Layer**:
```

```
  - `last_gnn_layer = Softmax_Layer(input_dim=hidden_dim, output_dim=outp
    - `input_dim=hidden_dim`: The softmax layer takes the output of the l
    - `output_dim=output_dim`: The output dimensionality is the number of
    - `name='sm_layer'`: As specified, names the softmax layer.
  - The softmax layer is appended to `self.layers`.

#### Embedding Method
- `H = X`: Initializes the activation matrix \( H \) as the input feature
- `for layer in self.layers[:-1]`: Iterates over all GNN layers, excludin
  - `H = layer.forward_pass(A, H)`: Passes the current activation \( H \)
    - Each GNN layer computes \( H = \tanh(A H W^T) \), updating \( H \)
- `return H`: Returns the final embedding (shape: `num_nodes, hidden_dim`

#### Forward Pass Method
- `H = self.embedding(A, X)`: Calls the `embedding` method to compute the
- `out = self.layers[-1].forward_pass(H)`: Passes the embeddings through
- `return out`: Returns the final output, the predicted probabilities for

### Assumptions
- `input_dim` is `num_nodes` (e.g., 34), matching the dimensionality of t
- `output_dim` is `num_classes` (e.g., 2 for the karate club's two affili
- `hidden_dim` is a hyperparameter (e.g., 16 or 32) specifying the size o
- `num_layers` is the number of GNN layers (excluding the softmax layer).
- `A` is `A_hat` (symmetrically normalized adjacency matrix from Step 2).
- `X` is the identity matrix from Step 3.
- `GNN_Layer` and `Softmax_Layer` are correctly implemented as provided e
- The `forward_pass` methods of both layers expect inputs with compatible

### Notes
- **Shapes**:
  - Input \( X \): `num_nodes, input_dim` (e.g., `34, 34`).
  - \( A \): `num_nodes, num_nodes` (e.g., `34, 34`).
  - GNN layer outputs: `num_nodes, hidden_dim` (e.g., `34, 16`).
  - Softmax layer output: `num_nodes, output_dim` (e.g., `34, 2`).
- **Loop Adjustment**: The loop in `__init__` was adjusted to `num_layers
- **Softmax Layer**: Assumes `Softmax_Layer.forward_pass` expects a singl

### Next Steps
Please run the provided code in your Jupyter notebook and share:
- Any errors or unexpected outputs.
- The next part of the notebook or additional TODOs to fill (e.g., traini
- If you need help debugging issues or adjusting parameters (e.g., `num_l
- If you have specific values for `num_nodes`, `num_classes`, or `hidden_
```



```
I'll assist with the next steps or any necessary corrections based on you
```

Now complete this code to instantiate the previous:
gnn_model = GNN(
  input_dim=??,
  output_dim=??,
  num_layers=2,
  hidden_dim=[16, 2],
)
gnn_model

Let's fill in the TODOs for instantiating the `GNN` class based on the provided context and previous steps. The `GNN` class requires `input_dim`, `output_dim`, `num_layers`, and `hidden_dim` to set up the multi-layer Graph Convolutional Network (GCN) with a softmax layer for the karate club classification task. We need to determine the appropriate values for `input_dim` and `output_dim` based on the problem context.

Context Recap

- **Dataset**: The karate club graph, which typically has `num_nodes = 34` nodes.
- **Input Features**: From Step 3, the input feature matrix X is the identity matrix of shape `num_nodes, num_nodes` (e.g., `34, 34`), as no explicit node features are provided.
- **Output Classes**: The task is to classify nodes into two groups (Officer and Mr. Hi), so `num_classes = 2`.
- **GNN Architecture**:
  - The `GNN` class creates `num_layers` GNN layers followed by a softmax layer.
  - The first GNN layer takes `input_dim` (the dimensionality of X) and outputs `hidden_dim[0]` (e.g., 16).
  - Subsequent GNN layers (if any) take `hidden_dim[i-1]` as input and output `hidden_dim[i]`.
  - The softmax layer takes the output of the last GNN layer (`hidden_dim[-1]`) and produces `output_dim` (number of classes).

- **Provided Parameters**:
  - `num_layers=2` : Indicates two GNN layers (plus the softmax layer).
  - `hidden_dim=[16, 2]` : Specifies the output dimensions of the two GNN layers (first layer outputs 16-dimensional embeddings, second layer outputs 2-dimensional embeddings).

Determining `input_dim` and `output_dim`

- **input_dim**: The input dimension is the dimensionality of the input feature matrix X. Since X is the identity matrix of shape `num_nodes, num_nodes` (e.g., `34, 34`), the input dimension is `num_nodes` (e.g., 34).
- **output_dim**: The output dimension is the number of classes for the classification task, which is `num_classes = 2` (for Officer and Mr. Hi).
- **hidden_dim**: The provided `hidden_dim=[16, 2]` means:
  - First GNN layer: `input_dim=num_nodes` (e.g., 34), `output_dim=16` .
  - Second GNN layer: `input_dim=16` , `output_dim=2` .
  - Softmax layer: `input_dim=2` (from the last GNN layer), `output_dim=num_classes=2` .

Code Implementation

Below is the completed code, filling in only the TODOs for `input_dim` and `output_dim` , wrapped in the required artifact tag.

gnn_instantiation.py
python

Edit in files • Show inline

```
text

### Explanation
- **input_dim=num_nodes**:
  - Set to `num_nodes` (e.g., 34 for the karate club), as the input featu
- **output_dim=num_classes**:
  - Set to `num_classes` (e.g., 2), as the final softmax layer outputs pr
```



```
- **num_layers=2**:
  - Specifies two GNN layers, as provided. The `GNN` class will create:
    - First GNN layer: `input_dim=num_nodes`, `output_dim=hidden_dim[0]=1
    - Second GNN layer: `input_dim=hidden_dim[0]=16`, `output_dim=hidden_
    - Softmax layer: `input_dim=hidden_dim[1]=2`, `output_dim=num_classes
- **hidden_dim=[16, 2]**:
  - Defines the output dimensions of the two GNN layers, as provided.
- **gnn_model**:
  - Instantiates the `GNN` class, creating a network with two GNN layers
  - The `__repr__` method will display the layers, e.g.:
```

GNN_Layer: W_layer_0(34, 16)

GNN_Layer: W_layer_1(16, 2)

Softmax_Layer: W_sm_layer(2, 2)

```
text

(assuming `num_nodes=34`, `num_classes=2`).

### Assumptions
- `num_nodes` is defined in the notebook's scope (e.g., 34 for the karate
- `num_classes` is defined (e.g., 2 for the two affiliations).
- `A_hat` and `X` from previous steps are compatible with the model (shap
- `GNN`, `GNN_Layer`, and `Softmax_Layer` are correctly implemented as pr
- `num_layers=2` and `hidden_dim=[16, 2]` are fixed as given, implying tw

### Potential Issues to Watch For
- **Variable Definitions**: Ensure `num_nodes` and `num_classes` are defi
- **Shape Compatibility**:
- `X` must have shape `num_nodes, num_nodes` (e.g., `34, 34`).
- `A_hat` must have shape `num_nodes, num_nodes`.
- The GNN output will be `num_nodes, 2`, and the softmax output will be `
- **Hidden Dimensions**: The `hidden_dim=[16, 2]` list must have length e
- **Optimizer Learning Rate**: The previous optimizer had `learning_rate=

### Next Steps
Please run the provided code in your Jupyter notebook and share:
- The output of `gnn_model` (e.g., the printed representation of the laye
- Any errors or unexpected results.
- The next part of the notebook or additional TODOs (e.g., training loop,
- If you need help verifying shapes, debugging, or adjusting parameters (

I'll assist with the next steps or any corrections based on your feedback
```

I am assuming there is a bug in the skeleton:
since hidden_dim is set to a list but treated as an integer in the class. I deleted the list and just passed in 16? Or is there a better solution

You've identified a potential issue in the provided `GNN` class skeleton, where the `hidden_dim` parameter is passed as a list (`[16, 2]`) but may be treated as an integer in the class implementation. Let's analyze the issue, confirm whether there's a bug, and provide the best solution, considering the context of the karate club GCN and the code provided.

Analyzing the Issue

- **Context**:
  - The `GNN` class constructs a network with `num_layers` GNN layers followed by a softmax layer.
  - The `hidden_dim` parameter is provided as a list `[16, 2]` with `num_layers=2` , suggesting that each GNN layer has a specific output dimension:
    - First GNN layer: output dimension = 16.
    - Second GNN layer: output dimension = 2.
  - The softmax layer takes the output of the last GNN layer (dimension 2) and produces `output_dim` (e.g., `num_classes=2`).
- **Skeleton Code**:
  - In the `__init__` method, the first GNN layer uses `output_dim=hidden_dim` , and subsequent GNN layers use `input_dim=hidden_dim` and `output_dim=hidden_dim` , which implies `hidden_dim` is treated as a single integer.
  - The softmax layer uses `input_dim=hidden_dim` , expecting the output dimension of the last GNN layer.
  - This is inconsistent with `hidden_dim` being a list, as the code does not index the list to assign different dimensions to each layer.
- **Your Solution**:

  - You suggest replacing `hidden_dim=[16, 2]` with a single integer (e.g., `16`) to make it consistent with how `hidden_dim` is used in the skeleton.
  - This would mean all GNN layers have the same output dimension (e.g., 16), and the softmax layer takes an input dimension of 16.
- **Potential Bug**:
  - The skeleton code appears to have a bug because it treats `hidden_dim` as an integer (e.g., `output_dim=hidden_dim`) but the instantiation passes a list (`[16, 2]`).
  - For `num_layers=2` , the intention seems to be that:
    - The first GNN layer outputs dimension 16 (`hidden_dim[0]`).
    - The second GNN layer outputs dimension 2 (`hidden_dim[1]`).
    - The softmax layer takes input dimension 2 and outputs `num_classes` .
  - However, the skeleton does not index `hidden_dim` (e.g., `hidden_dim[layer]`), causing a mismatch.

Evaluating Your Solution

- **Using `hidden_dim=16` :**
  - Setting `hidden_dim=16` (a single integer) would make the skeleton consistent, as all GNN layers would use `hidden_dim=16` for both input and output dimensions (except the first layer, which uses `input_dim=num_nodes` and `output_dim=16`).
  - The softmax layer would then have `input_dim=16` and `output_dim=num_classes` .
  - **Pros**:
    - Simplifies the code and avoids the list-indexing issue.
    - Works with the provided skeleton without modifying its logic.
  - **Cons**:
    - Loses the flexibility of having different hidden dimensions per layer (e.g., `[16, 2]`), which may be intentional for the karate club task.

- A hidden dimension of 16 for the last GNN layer means the softmax layer processes higher-dimensional inputs (16 instead of 2), which may not align with the task's goal of producing 2-dimensional embeddings before classification.
- May reduce model expressiveness, as the second GNN layer no longer reduces the dimensionality to 2, which could be useful for separating the two classes.

Better Solution

The best solution depends on the intended design of the GCN. Given the context (karate club classification with two classes and `hidden_dim=[16, 2]`), it's likely that the skeleton code has a bug, and the intention was to use different hidden dimensions for each layer. The `hidden_dim` list suggests a progressive reduction in dimensionality (from `num_nodes` to 16 to 2) before the softmax layer, which is common in GCNs for node classification to produce low-dimensional embeddings that align with the number of classes.

To fix the bug while preserving the intended architecture, we should modify the `GNN` class to index the `hidden_dim` list appropriately, ensuring each GNN layer uses the corresponding dimension from `hidden_dim`. Since you've asked to only fill in the TODOs in the instantiation and not modify the skeleton, I'll provide two approaches:

1. **Minimal Change (Your Approach)**: Use a single `hidden_dim` value (e.g., 16) to match the skeleton's current logic, requiring no changes to the class.
2. **Recommended Fix**: Modify the skeleton to handle `hidden_dim` as a list, but since you've specified not to change the skeleton, I'll suggest how to adapt the instantiation and explain the necessary class changes separately for clarity.

Approach 1: Minimal Change (Use `hidden_dim=16`)

Since the skeleton treats `hidden_dim` as an integer, we can set `hidden_dim=16` (taking the first value from `[16, 2]`) to make the instantiation compatible with the provided code. This assumes both GNN layers output 16-dimensional embeddings, and the softmax layer takes 16-dimensional inputs.

gnn_instantiation_fixed.py

python

Edit in files • Show inline

```
text

**Explanation**:
- **input_dim=num_nodes**: Matches the dimensionality of \( X \) (e.g., 3
- **output_dim=num_classes**: Matches the number of classes (e.g., 2 for
- **num_layers=2**: Specifies two GNN layers, as given.
- **hidden_dim=16**: Uses a single integer instead of the list `[16, 2]`,
- **Resulting Architecture**:
  - First GNN layer: `input_dim=num_nodes` (e.g., 34), `output_dim=16`.
  - Second GNN layer: `input_dim=16`, `output_dim=16`.
  - Softmax layer: `input_dim=16`, `output_dim=num_classes` (e.g., 2).
- **Output**: The `__repr__` will show:
```

GNN_Layer: W_layer_0(34, 16)

GNN_Layer: W_layer_1(16, 16)

Softmax_Layer: W_sm_layer(16, 2)

```
text

**Pros**:
- No changes to the skeleton code, adhering to your constraint.
- Simple and works immediately with the provided class.
- Maintains a consistent hidden dimension across GNN layers.

**Cons**:
- Ignores the second value in `hidden_dim=[16, 2]`, potentially deviating
- The softmax layer processes 16-dimensional inputs, which may be less ef

#### Approach 2: Recommended Fix (Handle `hidden_dim` as a List)
The intended architecture likely uses `hidden_dim=[16, 2]` to specify dif

If we assume the skeleton's bug is unintentional and the goal is to use `

Instead, let's acknowledge that using a single `hidden_dim` (e.g., 16 or

**Recommended Class Fix (For Reference, Not Applied)**:
If you were allowed to modify the skeleton, here's how the `__init__` sho

```python
```
```

```
class GNN():
  def __init__(self, input_dim, output_dim, hidden_dim, num_layers):
      self.input_dim = input_dim
      self.output_dim = output_dim
      self.hidden_dim = hidden_dim
      self.num_layers = num_layers

      self.layers = []
      # First GNN layer
      first_gnn_layer = GNN_Layer(input_dim=input_dim,
                                  output_dim=hidden_dim[0],
                                  name='layer_0')
      self.layers.append(first_gnn_layer)

      # Intermediate GNN layers
      for layer in range(num_layers - 1):
          gnn_temp = GNN_Layer(input_dim=hidden_dim[layer],
                               output_dim=hidden_dim[layer + 1],
                               name=f'layer_{layer + 1}')
          self.layers.append(gnn_temp)

      # Softmax layer
      last_gnn_layer = Softmax_Layer(input_dim=hidden_dim[-1],
                                     output_dim=output_dim,
                                     name='sm_layer')
      self.layers.append(last_gnn_layer)
```

Changes:

- Use `hidden_dim[0]` for the first layer's `output_dim`.
- Use `hidden_dim[layer]` for the input and `hidden_dim[layer + 1]` for the output of subsequent layers.
- Use `hidden_dim[-1]` for the softmax layer's `input_dim`.
- Add a check to ensure `len(hidden_dim) == num_layers`.

This would allow `hidden_dim=[16, 2]` to work as intended:

- First GNN layer: `input_dim=num_nodes` (e.g., 34), `output_dim=16`.
- Second GNN layer: `input_dim=16`, `output_dim=2`.
- Softmax layer: `input_dim=2`, `output_dim=num_classes` (e.g., 2).

Final Answer

Given the constraint not to modify the skeleton, the best instantiation is to use a single `hidden_dim` value, as you suggested. I've chosen `hidden_dim=16` to align with the first value in `[16, 2]`, as it's a reasonable hidden dimension size for a GCN and keeps the model expressive.

gnn_instantiation_fixed.py
python

Edit in files • Show inline

```
text

### Notes
- **Architecture**:
  - First GNN layer: `num_nodes` (e.g., 34) → 16.
  - Second GNN layer: 16 → 16.
  - Softmax layer: 16 → `num_classes` (e.g., 2).
- **Trade-offs**:
  - This matches the skeleton's logic but doesn't use `hidden_dim=[16, 2]
  - The softmax layer processes 16-dimensional inputs, which is less effi
- **Alternative**:
  - If you want the last GNN layer to output 2 dimensions (closer to `num
  - The recommended fix (modifying the skeleton) is ideal but violates th

### Next Steps
Please run the provided instantiation code and share:
- The output of `gnn_model` (e.g., the printed layers).
- Any errors or unexpected behavior.
- The next part of the notebook or additional TODOs (e.g., training, back
- If you're allowed to modify the skeleton or want to adopt the recommend
- Specific values for `num_nodes` (e.g., 34) or `num_classes` (e.g., 2) t

I'll assist with the next steps, debugging, or further adjustments based
```

Please use the better solution where we keep the list and change the previous code

Since you've requested to use the "better solution" where we keep `hidden_dim` as a list (`[16, 2]`) and modify the previous code, I understand you now permit changes to the `GNN` class skeleton to handle `hidden_dim` as a list correctly. This aligns with the intended architecture for the karate club GCN, where each GNN layer uses a specific output dimension from the `hidden_dim` list, allowing progressive dimensionality reduction (e.g., from `num_nodes` to 16 to 2) before the softmax layer.

The task is to:

1. Update the `GNN` class from the previous step (Question 6) to handle `hidden_dim` as a list, indexing it appropriately for each layer's output dimension.
2. Provide the instantiation code that uses `hidden_dim=[16, 2]` , as originally provided.

I'll revise the `GNN` class to fix the bug by indexing `hidden_dim` for each layer and ensure the instantiation matches the original call with `hidden_dim=[16, 2]` . Since you've asked to modify the "previous code" (the `GNN` class skeleton), I'll include both the updated class and the instantiation in a single artifact, ensuring all TODOs are filled and the code is consistent with the karate club GCN context.

Updated Code

Below is the revised `GNN` class with the TODOs filled and the logic adjusted to handle `hidden_dim` as a list, followed by the instantiation using `hidden_dim=[16, 2]` .

gnn_network.py
python

Edit in files • Show inline

```
text

## Explanation of Changes

#### `__init__` Method
- **First GNN Layer**:
  - `input_dim=input_dim`: Set to `num_nodes` (e.g., 34 for the karate cl
  - `output_dim=hidden_dim[0]`: Uses the first element of the `hidden_dim
  - `name='layer_0'`: As specified in the skeleton.
```



```
- **Intermediate GNN Layers**:
  - Loop runs for `num_layers - 1` iterations to create additional GNN la
  - `input_dim=hidden_dim[layer]`: The input dimension is the output dime
  - `output_dim=hidden_dim[layer + 1]`: The output dimension is the next
  - `name=f'layer_{layer + 1}''`: Names layers sequentially (e.g., `layer_
- **Softmax Layer**:
  - `input_dim=hidden_dim[-1]`: Uses the last element of `hidden_dim` (e.
  - `output_dim=output_dim`: Set to `num_classes` (e.g., 2 for Officer an
  - `name='sm_layer''`: As specified.
- **Validation**:
  - The code assumes `len(hidden_dim) == num_layers`. For `num_layers=2`,
  - If needed, you could add a check: `assert len(hidden_dim) == num_laye

#### `embedding` and `forward_pass` Methods
- These remain unchanged from the original skeleton, as they were correct
  - `embedding`: Iterates through all GNN layers (excluding the softmax l
  - `forward_pass`: Calls `embedding` to get the final GNN layer's output

#### Instantiation
- `input_dim=num_nodes`: Matches the dimensionality of \( X \) (e.g., 34)
- `output_dim=num_classes`: Matches the number of classes (e.g., 2).
- `num_layers=2`: Specifies two GNN layers, as provided.
- `hidden_dim=[16, 2]`: Uses the original list, with the first GNN layer
- **Resulting Architecture**:
  - First GNN layer: `num_nodes` (e.g., 34) → 16.
  - Second GNN layer: 16 → 2.
  - Softmax layer: 2 → `num_classes` (e.g., 2).
- **Output of `gnn_model`**:
  - The `__repr__` will display:
```

GNN_Layer: W_layer_0(34, 16)

GNN_Layer: W_layer_1(16, 2)

Softmax_Layer: W_sm_layer(2, 2)

```
text

(assuming `num_nodes=34`, `num_classes=2`).

### Why This Is the Better Solution
- **Preserves Intent**: The `hidden_dim=[16, 2]` list suggests a progress
- **Efficiency**: The final GNN layer outputs 2-dimensional embeddings, m
- **Flexibility**: Allows different hidden dimensions per layer, increasi
- **Fixes the Bug**: The original skeleton incorrectly treated `hidden_di
```

```
### Assumptions
- `num_nodes` is defined (e.g., 34 for the karate club).
- `num_classes` is defined (e.g., 2 for Officer and Mr. Hi).
- `GNN_Layer` and `Softmax_Layer` are implemented as provided in previous
- `A_hat` (symmetrically normalized adjacency matrix) and `X` (identity m
- `len(hidden_dim) == num_layers` (here, both are 2).

### Potential Issues to Watch For
- **List Length**: The code assumes `len(hidden_dim) == num_layers`. If `
```python
assert len(hidden_dim) == num_layers, "Length of hidden_dim must equal nu
```

- **Shapes**:
  - `X`: `num_nodes, num_nodes` (e.g., `34, 34`).
  - `A_hat`: `num_nodes, num_nodes`.
  - First GNN layer output: `num_nodes, 16`.
  - Second GNN layer output: `num_nodes, 2`.
  - Softmax output: `num_nodes, num_classes`.
- **Optimizer**: The previous optimizer had `learning_rate=0`, which prevents training. If training is next, ensure `learning_rate` is positive (e.g., 0.01).

Next Steps

Please run the provided code in your Jupyter notebook and share:

- The output of `gnn_model` (e.g., the printed layers).
- Any errors or unexpected behavior.
- The next part of the notebook or additional TODOs (e.g., training loop, backward pass, or evaluation).
- Specific values for `num_nodes` (e.g., 34) or `num_classes` (e.g., 2) to confirm compatibility.
- If you need help with training, debugging, or further adjustments (e.g., adding validation for `hidden_dim` length).

I'll assist with the next steps or any corrections based on your feedback!


```
IndexError                Traceback (most recent call last)
/tmp/ipython-input-1689888756.py in <cell line: 0>()
      1 # TODO
----> 2 gnn_model = GNN(
      3     input_dim=num_nodes,
      4     output_dim=num_classes,
      5     num_layers=2,

/tmp/ipython-input-2549675678.py in __init__(self, input_dim,
output_dim, hidden_dim, num_layers)
     21         for layer in range(num_layers - 1):
     22             gnn_temp = GNN_Layer(input_dim=hidden_dim[layer],
---> 23                                  output_dim=hidden_dim[layer + 1],
     24                                  name=f'layer_{layer + 1}')
     25             self.layers.append(gnn_temp)

IndexError: list index out of range
```

Thought for 3s

You're encountering an `IndexError: list index out of range` because of an issue in the loop indexing logic in the `GNN` class `__init__` method. Let's debug and fix this.

Root Cause of the Error

The error occurs in this line:

```python
for layer in range(num_layers - 1):
    gnn_temp = GNN_Layer(input_dim=hidden_dim[layer],
                         output_dim=hidden_dim[layer + 1],  # ← This caus
                         name=f'layer_{layer + 1}')
```

What's happening:

- `num_layers = 2`
- `hidden_dim = [16, 2]` (length = 2)
```

- The loop runs for `range(2 - 1) = range(1)`, so `layer = 0`
- For `layer = 0`:
  - `input_dim=hidden_dim[0] = 16` ✓ (valid)
  - `output_dim=hidden_dim[1] = 2` ✓ (valid)
- **Wait, that should work!** The error suggests the issue might be elsewhere.

Debugging the Issue

The error traceback shows the issue is at `hidden_dim[layer + 1]`, but logically this should work for the given parameters. The problem might be that:

1. **The loop is running more iterations than expected**, or
2. **`num_layers` or `hidden_dim` have different values** than we assume.

Let's add some debugging and fix the indexing logic more robustly.

Fixed Code

Here's the corrected `GNN` class with proper indexing and debugging to prevent the error:

python

```
class GNN():
    """This class leverages the GNN layer implemanted above by cascading
    def __init__(self,
                 input_dim,
                 output_dim,
                 hidden_dim,
                 num_layers,):

        # Validation
        assert len(hidden_dim) == num_layers, f"hidden_dim length ({len(h

        self.input_dim = input_dim
        self.output_dim = output_dim
        self.hidden_dim = hidden_dim
        self.num_layers = num_layers

        self.layers = []
```



```
        # First GNN layer: input_dim -> hidden_dim[0]
        first_gnn_layer = GNN_Layer(input_dim=input_dim,
                                    output_dim=hidden_dim[0],
                                    name='layer_0')
        self.layers.append(first_gnn_layer)

        # Intermediate GNN layers: hidden_dim[i-1] -> hidden_dim[i]
        for layer_idx in range(1, num_layers):  # Start from 1, go up to
            gnn_temp = GNN_Layer(input_dim=hidden_dim[layer_idx - 1],  #
                                 output_dim=hidden_dim[layer_idx],     #
                                 name=f'layer_{layer_idx}')
            self.layers.append(gnn_temp)

        # Softmax layer: hidden_dim[-1] -> output_dim
        last_gnn_layer = Softmax_Layer(input_dim=hidden_dim[-1],
                                       output_dim=output_dim,
                                       name='sm_layer')
        self.layers.append(last_gnn_layer)

    def __repr__(self):
        return '\n'.join([str(layer) for layer in self.layers])

    def embedding(self, A, X):
        H = X
        for layer in self.layers[:-1]:
            H = layer.forward_pass(A, H)
        return H

    def forward_pass(self, A, X):
        H = self.embedding(A, X)
        out = self.layers[-1].forward_pass(H)
        return out

# Instantiation
gnn_model = GNN(
    input_dim=num_nodes,
    output_dim=num_classes,
    num_layers=2,
    hidden_dim=[16, 2],
)
gnn_model
```

Key Changes

1. **Added Validation**:

```python
assert len(hidden_dim) == num_layers, f"hidden_dim length ({len(hidder
```

This ensures `hidden_dim` has the correct length before proceeding.

2. **Fixed Loop Indexing**:

```python
for layer_idx in range(1, num_layers):  # Start from 1, go up to num_l
    gnn_temp = GNN_Layer(input_dim=hidden_dim[layer_idx - 1],  # Outpu
                         output_dim=hidden_dim[layer_idx],     # Outpu
                         name=f'layer_{layer_idx}')
```

- Changed `layer` to `layer_idx` for clarity
- Loop runs from `1` to `num_layers-1` (exclusive)
- For `num_layers=2`, this runs once with `layer_idx=1`
- `input_dim=hidden_dim[0]` (16) and `output_dim=hidden_dim[1]` (2)

3. **Clearer Layer Naming**:

- Uses `layer_idx` directly in the name: `f'layer_{layer_idx}'`

How It Works Now

For `num_layers=2`, `hidden_dim=[16, 2]`:

1. **First GNN Layer** (layer 0):

- `input_dim=num_nodes` (e.g., 34)
- `output_dim=hidden_dim[0]=16`
- Name: `'layer_0'`

2. **Second GNN Layer** (layer 1):

- Loop runs once with `layer_idx=1`
- `input_dim=hidden_dim[0]=16`

- `output_dim=hidden_dim[1]=2`
- Name: `'layer_1'`

3. **Softmax Layer**:

- `input_dim=hidden_dim[-1]=2`
- `output_dim=num_classes` (e.g., 2)
- Name: `'sm_layer'`

Expected Output

The `gnn_model` should now print:

```text
GNN_Layer: W_layer_0(34, 16)
GNN_Layer: W_layer_1(16, 2)
Softmax_Layer: W_sm_layer(2, 2)
```

Debugging Steps (If Error Persists)

If you still get an error, please run this debugging code first to check your variables:

```python
# Debug: Check variable values
print(f"num_nodes: {num_nodes}")
print(f"num_classes: {num_classes}")
print(f"num_layers: 2")
print(f"hidden_dim: [16, 2]")
print(f"len(hidden_dim): {len([16, 2])}")
print(f"Expected: len(hidden_dim) == num_layers: {len([16, 2]) == 2}")

# Test the validation
try:
    assert len([16, 2]) == 2, f"hidden_dim length ({len([16, 2])}) must e
    print("Validation passed!")
except AssertionError as e:
    print(f"Validation failed: {e}")
```

Alternative: If You Want to Keep Original Loop Structure

If you prefer to keep the original loop structure (`range(num_layers - 1)`), here's the corrected version:

```python
# Alternative: Keep original loop structure
for layer in range(num_layers - 1):
    gnn_temp = GNN_Layer(input_dim=hidden_dim[layer],      # hidden_dim[0
                         output_dim=hidden_dim[layer + 1], # hidden_dim[1
                         name=f'layer_{layer + 1}')
    self.layers.append(gnn_temp)
```

This should also work for `num_layers=2` , but the `range(1, num_layers)` approach is clearer.

Next Steps

1. **Run the fixed code** above with the validation and corrected indexing.
2. **Share the output** of `gnn_model` to confirm it works.
3. **If you still get an error**, share:
   - The values of `num_nodes` and `num_classes` in your notebook
   - The exact error message and line number
   - The output of the debugging code above

The fix should resolve the `IndexError` . Let me know if you need further assistance!

This solved the problem. We now instantiate
test_nodes = np.array([0, 1, 8, 3, 8, 15, 16, 20, 25, 28, 30])
train_nodes = train_test_split(test_nodes, labels)

print(f'The nodes that will be used for training are:\n {train_nodes}')

print(f'The nodes that will be used for test/val are:\n {test_nodes}')
training_optim = Grad_Descent_Optimizer(learning_rate=2e-2,
weight_decay=2.5e-2)

We need to fill in the train function:


```
def train(model,
          A_hat,
          X,
          y_true,
          nodes_to_be_used_for_training,
          nodes_to_be_used_for_testing,
          threshold_value=.5,
          ealy_stopping_steps=60,
          num_epochs=20000):
  """trains our gnn model"""

  accs = []
  training_losses = []
  testing_losses = []
  embeddings = []
  y_preds = []

  minimum_loss = 1e7
  ealy_stopping_counter = 0

  for epoch in range(num_epochs):
    # TODO
    # y_pred = ??
    training_optim(y_pred, y_true, nodes_to_be_used_for_training)

    for layer in reversed(model.layers):
      layer.backward_pass(training_optim, need_update=True)

    embeddings.append(model.embedding(A_hat, X))
    y_preds.append(y_pred)
    acc_temp = (np.argmax(y_pred, axis=1) == np.argmax(y_true,
axis=1))[
        [i for i in range(y_true.shape[0]) if i not in
nodes_to_be_used_for_training]
```

```
    ]
    accs.append(np.mean(acc_temp))

    # loss_temp = ??
    # train_loss_temp = ??
    # test_loss_temp =??

    training_losses.append(train_loss_temp)
    testing_losses.append(test_loss_temp)

    if test_loss_temp < minimum_loss:
      minimum_loss = test_loss_temp
      ealy_stopping_counter = 0
    else:
      ealy_stopping_counter += 1

    if ealy_stopping_counter > ealy_stopping_steps:
      print("Training Stopped due to Early stopping!")
      break

    if epoch % 100 == 0:
      print(f"epoch #: {epoch+1} | train Loss: {train_loss_temp:.3f} | test
Loss: {test_loss_temp:.3f}")

  training_losses = np.array(training_losses)
  testing_losses = np.array(testing_losses)
  y_preds = threshold(np.array(y_preds), threshold_value)
  return training_losses, testing_losses, accs, embeddings, y_preds
```

Let's fill in the TODOs for the `train` function to train the GNN model for the karate club classification task. The function trains the GNN model using the provided `GNN` class, `A_hat` (symmetrically normalized adjacency matrix), `X` (identity matrix), `y_true` (ground-truth labels), and specified training and testing nodes. We need to implement the forward pass, compute the loss for both training and testing nodes, and handle the training loop with early stopping.

Context Recap

- **Model**: The `GNN` class with two GNN layers (`input_dim=num_nodes` → 16 → 2) and a softmax layer (2 → `num_classes=2`).
- **Inputs**:
  - `A_hat` : Symmetrically normalized adjacency matrix (shape: `num_nodes, num_nodes` , e.g., `34, 34`).
  - `X` : Identity matrix (shape: `num_nodes, num_nodes`).
  - `y_true` : Ground-truth labels, likely one-hot encoded (shape: `num_nodes, num_classes` , e.g., `34, 2`).
  - `nodes_to_be_used_for_training` : Indices of training nodes (e.g., from `train_nodes`).
  - `nodes_to_be_used_for_testing` : Indices of test/validation nodes (e.g., `test_nodes = [0, 1, 8, 3, 8, 15, 16, 20, 25, 28, 30]`).
- **Optimizer**: `Grad_Descent_Optimizer` with `learning_rate=2e-2` and `weight_decay=2.5e-2` (equivalent to 2λ for L2 regularization).
- **Task**: Semi-supervised node classification, where only some nodes (`train_nodes`) have labels used for training, and we predict labels for `test_nodes` .
- **TODOs**:
  - Compute `y_pred` using the model's forward pass.
  - Compute the total loss (`loss_temp`).
  - Compute the training loss (`train_loss_temp`) for training nodes.
  - Compute the testing loss (`test_loss_temp`) for testing nodes.

Loss Function

The `Softmax_Layer` likely outputs class probabilities (shape: `num_nodes, num_classes`). For classification, we typically use cross-entropy loss. The loss for a node i is:

$$\text{Loss}_i = -\sum_{c=1}^{\text{num\_classes}} y_{\text{true},i,c} \log(y_{\text{pred},i,c})$$

- **Training Loss**: Average cross-entropy loss over `nodes_to_be_used_for_training`.
- **Testing Loss**: Average cross-entropy loss over `nodes_to_be_used_for_testing`.
- **Total Loss**: Often the training loss, as it's used for optimization.

We'll assume the `Grad_Descent_Optimizer` computes the cross-entropy loss internally when called with `y_pred` and `y_true` (based on the call `training_optim(y_pred, y_true, nodes_to_be_used_for_training)`), but we need to compute `train_loss_temp` and `test_loss_temp` explicitly for logging.

Implementation

Below is the completed `train` function, filling in only the TODOs, wrapped in the required artifact tag.

train_gnn.py
python

Edit in files • Show inline

```
text

### Explanation of TODOs

#### Forward Pass
- **y_pred = model.forward_pass(A_hat, X)**:
  - Calls the `forward_pass` method of the `GNN` model, which:
    - Computes embeddings through all GNN layers: \( H = \tanh(\tilde{A}^
    - Passes the final embeddings (shape: `num_nodes, hidden_dim[-1]=2`)
  - `A_hat` is the symmetrically normalized adjacency matrix (shape: `num
  - `X` is the identity matrix (shape: `num_nodes, num_nodes`).
  - The output `y_pred` contains class probabilities for each node.

#### Loss Computation
- **loss_temp**:
  - Computes the per-node cross-entropy loss: \( -\sum_{c} y_{\text{true}
  - `np.log(np.clip(y_pred, 1e-10, 1.0))`: Applies log to `y_pred`, clipp
  - `y_true * np.log(...)`: Element-wise multiplication with ground-truth
  - `-np.sum(..., axis=1)`: Sums over the class dimension to get per-node
- **train_loss_temp**:
  - `loss_temp[nodes_to_be_used_for_training]`: Selects losses for traini
  - `np.mean(...)`: Computes the average loss over training nodes.
```



```
- **test_loss_temp**:
  - `loss_temp[nodes_to_be_used_for_testing]`: Selects losses for test no
  - `np.mean(...)`: Computes the average loss over test nodes.

### Assumptions
- **`y_true`**: One-hot encoded labels (shape: `num_nodes, num_classes`,
- **`nodes_to_be_used_for_training`**: Array of indices (e.g., from `trai
- **`nodes_to_be_used_for_testing`**: Array of indices (e.g., `[0, 1, 8,
- **`training_optim`**: The `Grad_Descent_Optimizer` computes the loss in
- **Shapes**:
  - `A_hat`: `num_nodes, num_nodes` (e.g., `34, 34`).
  - `X`: `num_nodes, num_nodes`.
  - `y_pred`: `num_nodes, num_classes`.
  - `y_true`: `num_nodes, num_classes`.
  - `loss_temp`: `num_nodes`.
- **threshold**: A function (defined elsewhere) that thresholds `y_preds`

### Notes
- **Loss Clipping**: `np.clip(y_pred, 1e-10, 1.0)` prevents numerical iss
- **Optimizer**: The call `training_optim(y_pred, y_true, nodes_to_be_use
- **Early Stopping**: The function stops if the test loss doesn't improve
- **Typo**: The parameter `ealy_stopping_steps` should likely be `early_s

### Potential Issues to Watch For
- **Duplicate Test Nodes**: `test_nodes` contains duplicates (e.g., `8` a
- **train_test_split**: The call `train_nodes = train_test_split(test_nod
  ```python
  train_nodes = np.setdiff1d(np.arange(num_nodes), test_nodes)
```

- **Optimizer Learning Rate**: `learning_rate=2e-2` and `weight_decay=2.5e-2` are reasonable, but ensure they work well for convergence.
- **y_true Format**: Assumes one-hot encoded labels. If `y_true` is a vector of class indices, adjust the accuracy computation or convert to one-hot:

```
python

y_true_one_hot = np.eye(num_classes)[y_true]
```

- **Numerical Stability**: If `y_pred` contains values exactly 0 or 1, the clipping in `np.log` handles it, but monitor for NaNs.

Debugging Code
```

If you encounter issues, run this to check shapes and values:

```python
print(f"A_hat shape: {A_hat.shape}")
print(f"X shape: {X.shape}")
print(f"y_true shape: {y_true.shape}")
print(f"Training nodes: {nodes_to_be_used_for_training}")
print(f"Testing nodes: {nodes_to_be_used_for_testing}")
print(f"y_pred shape: {model.forward_pass(A_hat, X).shape}")
```

Next Steps

Please run the provided `train` function and share:

- The output (e.g., printed losses every 100 epochs, final losses, or early stopping message).
- Any errors or unexpected behavior (e.g., NaNs, shape mismatches).
- The next part of the notebook or additional TODOs (e.g., evaluation, plotting losses).
- Values of `num_nodes`, `num_classes`, or the output of the debugging code above.
- If `train_test_split` or `threshold` functions need clarification.

I'll assist with debugging or further steps based on your feedback!

> AFter training the model, we get 100 percent accuracy on the test data. However, we see two samples that are misclassified
>
> Q. 9. Explain why we obtain a 100% on accuracy on our test set, yet we see in the plot above that 2 samples seem to be misclassified.

The observation that your GNN model achieves 100% accuracy on the test set, yet a plot shows two samples as misclassified, suggests a potential inconsistency or misunderstanding in how accuracy is computed, how the test set is defined, or how the plot is generated. Let's analyze the possible reasons for this discrepancy and explain why this might occur, given the context of the karate club GCN classification task.

Context Recap

- **Task**: Semi-supervised node classification on the karate club graph (typically 34 nodes) to classify nodes into two classes (Officer and Mr. Hi).
- **Test Set**: Defined by `test_nodes = np.array([0, 1, 8, 3, 8, 15, 16, 20, 25, 28, 30])`, which has 11 elements but includes a duplicate (`8` appears twice). Thus, the unique test set likely has 10 nodes (`np.unique(test_nodes) = [0, 1, 3, 8, 15, 16, 20, 25, 28, 30]`).
- **Training**: The `train` function computes accuracy on nodes not used for training (`acc_temp = (np.argmax(y_pred, axis=1) == np.argmax(y_true, axis=1))[not_training_nodes]`), which likely includes the test nodes.
- **Accuracy**: Reported as 100% on the test set, meaning all test nodes are correctly classified according to the accuracy metric.
- **Plot**: Shows two samples as misclassified, indicating a visual representation (e.g., a scatter plot of node embeddings) suggests two nodes are incorrectly placed (e.g., in the wrong cluster or class region).

Why 100% Accuracy but Two Misclassified Samples in the Plot?

Here are the most likely explanations for this discrepancy:

1. Accuracy is Computed on Unique Test Nodes, but the Plot Includes Duplicates

- **Issue**: The `test_nodes` array contains a duplicate (`8` appears twice). If the accuracy computation uses unique nodes (`np.unique(test_nodes)`), it evaluates only 10 nodes, and all are correctly classified (100% accuracy). However, the plot may include the duplicate node (`8`) twice, and one of its representations (or another node's) appears misclassified in the visualization.
- **Explanation**:
  - In the `train` function, accuracy is computed as:

```python
acc_temp = (np.argmax(y_pred, axis=1) == np.argmax(y_true, axis=1))
    [i for i in range(y_true.shape[0]) if i not in nodes_to_be_used
]
```

This evaluates accuracy on non-training nodes, likely including `test_nodes`. If `test_nodes` is processed as unique nodes in the accuracy calculation (e.g., due to `np.unique` or set operations in `train_test_split`), the duplicate node `8` is counted once, and all 10 unique test nodes are correctly classified.

- The plot (likely showing node embeddings from `model.embedding(A_hat, X)`, shape: `num_nodes, 2`) may display all 11 test node indices, including the duplicate `8`. If node `8` is correctly classified but visually appears in an ambiguous region (or another node is misclassified in the plot), it could create the appearance of two misclassified samples.

- **Why Two Misclassifications?**:
  - The plot may show node `8` twice, with one instance appearing in the wrong cluster due to visualization artifacts (e.g., overlapping points or projection issues).
  - Alternatively, two other nodes in the test set are visually misplaced in the embedding space but still correctly classified (e.g., their `y_pred` probabilities assign the correct class despite their embedding positions).

- **Solution**:
  - Ensure `test_nodes` is unique before training:

```python
test_nodes = np.unique(test_nodes)  # [0, 1, 3, 8, 15, 16, 20, 25,
```

  - Check the plot generation code to confirm it uses unique test nodes or labels duplicates correctly.
  - Verify the number of test nodes used in accuracy computation:

```python
print(f"Number of unique test nodes: {len(np.unique(test_nodes))}")
print(f"Test nodes: {np.unique(test_nodes)}")
```

2. Accuracy is Correct, but the Plot Reflects Embedding Misplacement

- **Issue**: The 100% accuracy indicates that all test nodes are correctly classified based on `np.argmax(y_pred, axis=1)` matching `np.argmax(y_true, axis=1)`. However, the plot (likely a 2D scatter plot of embeddings from `model.embedding(A_hat, X)`) shows two nodes in the wrong cluster or region, suggesting their embeddings are closer to the opposite class's region.

- **Explanation**:
  - The GNN's final layer outputs 2D embeddings (shape: `num_nodes, 2`), which are passed through the softmax layer to produce `y_pred`. The softmax layer can correctly classify nodes even if their embeddings are not perfectly separated in the 2D space, as long as the probability for the correct class is higher.
  - For example, a node's embedding might be near the decision boundary or in the "wrong" cluster in the plot, but the softmax output still assigns the correct class (e.g., `[0.51, 0.49]` for class 0).
  - The plot visualizes the embeddings (`embeddings[-1]`, shape: `num_nodes, 2`), not the final probabilities, so two nodes may appear misclassified due to their embedding positions, even though their predicted labels are correct.

- **Why Two Misclassifications?**:
  - Two test nodes have embeddings that are not well-separated in the 2D space (e.g., near the decision boundary or in the opposite class's cluster) but are still correctly classified by the softmax layer.
  - This is common in semi-supervised GCNs, where embeddings are learned based on graph structure and training labels, and test nodes near high-degree nodes or with ambiguous connections may have less distinct embeddings.

- **Solution**:
  - Inspect the embeddings for test nodes:

```python
final_embeddings = model.embedding(A_hat, X)  # Shape: (num_nodes,
test_embeddings = final_embeddings[test_nodes]
print(f"Test node embeddings:\n{test_embeddings}")
```

```
print(f"Predicted labels: {np.argmax(y_pred[test_nodes], axis=1)}")
print(f"True labels: {np.argmax(y_true[test_nodes], axis=1)}")
```

- Check if the plot uses `embeddings[-1]` correctly and whether it highlights test nodes distinctly.
- Consider adjusting the model (e.g., increasing `hidden_dim` or training epochs) to improve embedding separation, though 100% accuracy suggests this isn't necessary for classification.

3. Accuracy is Computed on Non-Training Nodes, Not Just Test Nodes

- **Issue**: The accuracy in the `train` function is computed on all nodes *not in* `nodes_to_be_used_for_training`, which may include more nodes than `nodes_to_be_used_for_testing`. If the test set (`test_nodes`) has misclassifications, but other non-training nodes are correctly classified, the average accuracy could still be reported as 100% due to a larger set of nodes.
- **Explanation**:
  - The accuracy line:

    ```
    python

    acc_temp = (np.argmax(y_pred, axis=1) == np.argmax(y_true, axis=1)
        [i for i in range(y_true.shape[0]) if i not in nodes_to_be_used
    ]
    ```

    evaluates all non-training nodes, which could include validation nodes or unlabeled nodes in addition to `test_nodes`.
  - Suppose `num_nodes=34`, `len(test_nodes)=10` (unique), and `train_nodes` covers the remaining nodes (e.g., 24). If `test_nodes` has two misclassifications (80% accuracy), but the other 14 non-training nodes are all correct, the overall accuracy could be inflated:

$$\text{Accuracy} = \frac{8 + 14}{10 + 14} = \frac{22}{24} \approx 91.67\%$$

    However, if the code or output is misreporting this as 100%, there might be a bug in the accuracy computation or output.

  - The plot specifically highlights `test_nodes`, showing two misclassifications, while the reported accuracy includes additional nodes.
- **Why Two Misclassifications?**:
  - Two test nodes are misclassified (e.g., nodes `0` and `1` have incorrect `np.argmax(y_pred)`), but the accuracy metric averages over a larger set, masking the errors.
- **Solution**:
  - Compute accuracy specifically for `test_nodes`:

    ```
    python

    test_acc = np.mean((np.argmax(y_pred[test_nodes], axis=1) == np.arg
    print(f"Test accuracy: {test_acc:.3f}")
    ```
  - Check which nodes are included in `acc_temp`:

    ```
    python

    non_training_nodes = [i for i in range(y_true.shape[0]) if i not in
    print(f"Non-training nodes: {non_training_nodes}")
    print(f"Test nodes: {test_nodes}")
    ```
  - If `non_training_nodes` includes more than `test_nodes`, adjust the accuracy computation to focus on `test_nodes`.

4. Plotting Error or Visualization Artifact

- **Issue**: The plot (likely a scatter plot of 2D embeddings) incorrectly labels or positions two test nodes, creating the appearance of misclassification, while the actual predictions (`y_pred`) are correct.
- **Explanation**:
  - The plot may use `embeddings[-1]` (shape: `num_nodes, 2`) colored by predicted or true labels. Errors in the plotting code (e.g., incorrect indexing, color mapping, or handling of duplicates) could show two nodes as misclassified.

- For example, if the plot uses `test_nodes` with duplicates or mislabels points, it could show node `8` or others in the wrong cluster.

- **Why Two Misclassifications?**:
  - A plotting bug (e.g., misaligned labels or duplicate points) highlights two nodes incorrectly.
  - The embeddings may be correct, but the visualization misrepresents them.
- **Solution**:
  - Share the plotting code to verify how test nodes are visualized.
  - Example plotting code (assuming Matplotlib):

```python
import matplotlib.pyplot as plt
embeddings = model.embedding(A_hat, X)  # Shape: (num_nodes, 2)
plt.scatter(embeddings[:, 0], embeddings[:, 1], c=np.argmax(y_true,
plt.scatter(embeddings[test_nodes, 0], embeddings[test_nodes, 1], c
plt.legend()
plt.show()
```

  - Ensure test nodes are uniquely plotted and colored by predicted labels (`np.argmax(y_pred[test_nodes], axis=1)`).

5. Thresholding Issue in Final Predictions

- **Issue**: The `train` function returns `y_preds = threshold(np.array(y_preds), threshold_value=0.5)`. If the plot uses the thresholded predictions, but the accuracy uses raw `y_pred`, a mismatch in thresholding could cause apparent misclassifications.
- **Explanation**:
  - The accuracy uses `np.argmax(y_pred, axis=1)`, which takes the class with the highest probability.
  - The `threshold` function (undefined in the provided code) may convert probabilities to binary labels differently (e.g., `y_pred[:, 1] > 0.5` for class 1). If two test nodes have probabilities close to 0.5, thresholding might assign them incorrectly in the plot, while `np.argmax` is correct.

- **Why Two Misclassifications?**:
  - Two test nodes have probabilities near the threshold (e.g., `[0.51, 0.49]`), causing `threshold` to assign the wrong class in the plot, but `np.argmax` assigns the correct class.
- **Solution**:
  - Check the `threshold` function definition.
  - Compare raw and thresholded predictions:

```python
print(f"Raw predicted labels: {np.argmax(y_preds[-1][test_nodes], a
print(f"Thresholded predicted labels: {threshold(y_preds[-1][test_n
print(f"True labels: {np.argmax(y_true[test_nodes], axis=1)}")
```

Most Likely Explanation

The most likely cause is **Option 1 (Duplicate Test Nodes)** or **Option 2 (Embedding Misplacement)**:

- **Duplicate Nodes**: The duplicate `8` in `test_nodes` causes the plot to show node `8` twice, with one instance appearing misclassified, or another node's embedding is visually ambiguous.
- **Embedding Misplacement**: Two test nodes have embeddings near the decision boundary, appearing misclassified in the plot, but their softmax probabilities yield correct classifications.

Recommended Actions

1. **Fix Duplicate Test Nodes**:

```python
test_nodes = np.unique(test_nodes)  # Ensure unique test nodes
train_nodes = np.setdiff1d(np.arange(num_nodes), test_nodes)  # Recalc
print(f"Unique test nodes: {test_nodes}")
print(f"Train nodes: {train_nodes}")
```

2. **Inspect Test Node Predictions and Embeddings**:

```python
y_pred = model.forward_pass(A_hat, X)
embeddings = model.embedding(A_hat, X)
test_pred_labels = np.argmax(y_pred[test_nodes], axis=1)
test_true_labels = np.argmax(y_true[test_nodes], axis=1)
test_embeddings = embeddings[test_nodes]
print(f"Test node indices: {test_nodes}")
print(f"Predicted labels: {test_pred_labels}")
print(f"True labels: {test_true_labels}")
print(f"Test embeddings:\n{test_embeddings}")
```

3. **Verify Plotting Code**: Share the code generating the plot. Ensure it uses `test_nodes` correctly and colors points by predicted labels (`np.argmax(y_pred[test_nodes], axis=1)`).
4. **Check Accuracy Computation**: Ensure accuracy is computed only on `test_nodes`:

```python
test_acc = np.mean(np.argmax(y_pred[test_nodes], axis=1) == np.argmax(
print(f"Test accuracy: {test_acc:.3f}")
```

5. **Examine Thresholding**: If the plot uses `y_preds` after `threshold`, check:

```python
final_y_pred = threshold(y_preds[-1][test_nodes], threshold_value=0.5)
print(f"Thresholded predictions: {final_y_pred}")
```

Next Steps

Please provide:

- The plotting code used to visualize the embeddings and misclassified samples.
- Output of the debugging code above (e.g., test node predictions, embeddings, and accuracy).
- Values of `num_nodes` (e.g., 34) and `num_classes` (e.g., 2).
- The definition of the `threshold` function or how `y_preds` is processed for plotting.

- Confirmation of whether `train_test_split` removes `test_nodes` correctly to form `train_nodes`.

This will help pinpoint whether the issue is due to duplicates, embedding visualization, or a mismatch in accuracy computation. I'll provide a tailored solution based on your feedback!

> I made a mistake. The two incorrect classifications are shown on the resulting graph of all the nodes, both testing and training. So what is the reason it has 100% on test data but not 100% on all the data

Thank you for clarifying that the two misclassified samples appear in a plot of *all nodes* (both training and testing) rather than just the test nodes, while the reported accuracy is 100% on the test set. This changes the analysis, as it indicates that the misclassifications are among the entire set of nodes (training and testing), but the test set specifically achieves perfect accuracy. Let's analyze why this occurs in the context of the karate club GCN classification task and provide a clear explanation.

Context Recap

- **Task**: Semi-supervised node classification on the karate club graph (typically 34 nodes) to classify nodes into two classes (Officer and Mr. Hi).
- **Test Set**: Defined by `test_nodes = np.unique([0, 1, 8, 3, 8, 15, 16, 20, 25, 28, 30])`, which has 10 unique nodes after removing duplicates (`[0, 1, 3, 8, 15, 16, 20, 25, 28, 30]`).
- **Training Set**: `train_nodes = np.setdiff1d(np.arange(num_nodes), test_nodes)`, likely 24 nodes if `num_nodes=34`.
- **Model**: A GNN with two layers (`num_nodes → 16 → 2`) followed by a softmax layer (`2 → num_classes=2`).
- **Accuracy**: The `train` function computes accuracy on non-training nodes (likely including `test_nodes`) and reports 100% accuracy on the test set, meaning all test nodes are correctly classified.
- **Plot**: A visualization (likely a 2D scatter plot of embeddings from `model.embedding(A_hat, X)`, shape: `num_nodes, 2`) shows all nodes, with two nodes (from either training or test sets) appearing misclassified.

- **Training**: Uses `Grad_Descent_Optimizer` with `learning_rate=2e-2`, `weight_decay=2.5e-2`, and cross-entropy loss on training nodes, with early stopping based on test loss.

Why 100% Accuracy on Test Data but Two Misclassifications in the Plot of All Nodes?

The key observation is that the test set (`test_nodes`) achieves 100% accuracy, but when visualizing all nodes (training and testing), two nodes appear misclassified in the plot. Here are the most likely explanations:

1. Misclassifications Are in the Training Set

- **Explanation**:
  - The 100% test accuracy means all nodes in `test_nodes` (e.g., 10 nodes) are correctly classified: `np.argmax(y_pred[test_nodes], axis=1) == np.argmax(y_true[test_nodes], axis=1)`.
  - The plot shows all nodes (e.g., 34 nodes), and two nodes appear misclassified (e.g., their embeddings are in the wrong cluster or their predicted labels don't match their true labels).
  - Since the test nodes are correctly classified, the two misclassified nodes must be in the training set (`train_nodes`, e.g., 24 nodes).
  - In semi-supervised learning, the GNN optimizes the loss only on the training nodes (`training_optim(y_pred, y_true, nodes_to_be_used_for_training)`), but the model's predictions depend on the graph structure (`A_hat`) and message passing, which can affect all nodes. Some training nodes may have ambiguous connections (e.g., nodes with edges to both Officer and Mr. Hi groups), leading to incorrect predictions despite training.
- **Why Two Misclassifications?**:
  - Two training nodes have embeddings or predicted probabilities that result in incorrect class assignments (e.g., their `y_pred` probabilities favor the wrong class). This could happen if:
    - These nodes are near the boundary between the two classes in the graph (e.g., high-degree nodes connected to both groups).

    - The model's capacity (e.g., `hidden_dim=[16, 2]`) or training duration (up to 20,000 epochs with early stopping) is insufficient to perfectly fit all training nodes.
  - The plot (likely showing `model.embedding(A_hat, X)`) visualizes these nodes in the wrong cluster or colors them by predicted labels, highlighting the misclassifications.
- **Evidence**:
  - The `train` function computes accuracy on non-training nodes:

```python
acc_temp = (np.argmax(y_pred, axis=1) == np.argmax(y_true, axis=1)
    [i for i in range(y_true.shape[0]) if i not in nodes_to_be_used
]
```

    If this includes only `test_nodes` (or a superset), and all test nodes are correct, the 100% accuracy reflects the test set performance.
  - The training loss is computed only on `train_nodes`, so the model may not perfectly classify all training nodes, especially in a semi-supervised setting where graph structure influences predictions.
- **Solution**:
  - Compute accuracy on training nodes to confirm misclassifications:

```python
y_pred = model.forward_pass(A_hat, X)
train_acc = np.mean(np.argmax(y_pred[train_nodes], axis=1) == np.a
print(f"Training accuracy: {train_acc:.3f}")
```

  - Identify misclassified nodes:

```python
misclassified = np.where(np.argmax(y_pred, axis=1) != np.argmax(y_
print(f"Misclassified nodes: {misclassified}")
```

```
print(f"Are they in training set? {[i in train_nodes for i in misc
print(f"Are they in test set? {[i in test_nodes for i in misclassi
```

- If the misclassified nodes are in `train_nodes`, this confirms the issue. You could increase model capacity (e.g., adjust `hidden_dim`) or training epochs, but 100% test accuracy suggests the model is already effective for the test set.

2. Plot Reflects Embeddings, Not Final Predictions

- **Explanation**:
  - The plot likely shows the 2D embeddings from `model.embedding(A_hat, X)` (shape: `num_nodes, 2`), colored by predicted or true labels. Two nodes appear in the wrong cluster (e.g., their embeddings are closer to the opposite class's region), but their final predictions (`y_pred` from the softmax layer) are correct for the test nodes.
  - In a GCN, the embeddings (before the softmax layer) are influenced by the graph structure and may not perfectly align with class boundaries, especially for training nodes with complex connections. The softmax layer can still produce correct probabilities for test nodes due to learned weights, leading to 100% test accuracy.
  - The two misclassified nodes in the plot are likely training nodes whose embeddings are not well-separated, but this doesn't affect test accuracy.
- **Why Two Misclassifications?**:
  - Two training nodes have embeddings near the decision boundary or in the wrong cluster due to the graph's topology (e.g., nodes with edges to both classes). The softmax layer corrects predictions for test nodes, but these training nodes are misclassified.
- **Solution**:
  - Compare embeddings and predictions:

```python
embeddings = model.embedding(A_hat, X)  # Shape: (num_nodes, 2)
y_pred = model.forward_pass(A_hat, X)  # Shape: (num_nodes, num_cla
misclassified = np.where(np.argmax(y_pred, axis=1) != np.argmax(y_t
```



```
print(f"Misclassified nodes: {misclassified}")
print(f"Embeddings of misclassified nodes:\n{embeddings[misclassif
```

  - Check the plotting code to ensure it uses predicted labels (`np.argmax(y_pred, axis=1)`) rather than embeddings alone. Example:

```python
import matplotlib.pyplot as plt
plt.scatter(embeddings[:, 0], embeddings[:, 1], c=np.argmax(y_pred,
plt.scatter(embeddings[test_nodes, 0], embeddings[test_nodes, 1], c
plt.legend()
plt.show()
```

  - If the plot colors nodes by true labels or embeddings, adjust it to use predicted labels.

3. Accuracy Computation Includes More Than Test Nodes

- **Explanation**:
  - The `train` function computes accuracy on non-training nodes:

```python
acc_temp = (np.argmax(y_pred, axis=1) == np.argmax(y_true, axis=1)
    [i for i in range(y_true.shape[0]) if i not in nodes_to_be_usec
]
```

    This may include nodes beyond `test_nodes` (e.g., validation nodes or unlabeled nodes). If `num_nodes=34`, `len(train_nodes)=24`, and `len(test_nodes)=10`, the non-training nodes may equal `test_nodes`. However, if additional nodes are included, 100% accuracy on this set doesn't guarantee 100% on `test_nodes` alone.
  - The plot shows all nodes, and two training nodes are misclassified, reducing the overall accuracy but not affecting the reported test accuracy.
- **Why Two Misclassifications?**:
  - The two misclassified nodes are in `train_nodes`, and the test set is perfectly classified.

- **Solution**:
  - Compute test accuracy explicitly:

```python
test_acc = np.mean(np.argmax(y_pred[test_nodes], axis=1) == np.argm
print(f"Test accuracy: {test_acc:.3f}")
```

  - Compare with non-training accuracy:

```python
non_training_nodes = [i for i in range(y_true.shape[0]) if i not in
non_training_acc = np.mean(np.argmax(y_pred[non_training_nodes], ax
print(f"Non-training accuracy: {non_training_acc:.3f}")
```

4. Plotting Error or Visualization Artifact

- **Explanation**:
  - The plot may incorrectly label or position two nodes due to a bug in the plotting code (e.g., wrong color mapping, indexing errors, or duplicate nodes). If the plot uses `y_pred` or embeddings incorrectly, it could show two nodes as misclassified even though predictions are correct for test nodes.
  - The 100% test accuracy is based on `y_pred`, which is correct for `test_nodes`.
- **Why Two Misclassifications?**:
  - A plotting error (e.g., coloring by true labels instead of predicted labels, or mishandling node indices) shows two nodes in the wrong cluster.
- **Solution**:
  - Share the plotting code to verify. Example correct plotting:

```python
import matplotlib.pyplot as plt
embeddings = model.embedding(A_hat, X)
plt.scatter(embeddings[:, 0], embeddings[:, 1], c=np.argmax(y_pred,
plt.scatter(embeddings[test_nodes, 0], embeddings[test_nodes, 1], c
```



```python
plt.legend()
plt.show()
```

Most Likely Explanation

The most likely cause is **Option 1: Misclassifications in the Training Set**:

- The 100% test accuracy indicates all test nodes (`test_nodes`) are correctly classified (`np.argmax(y_pred[test_nodes], axis=1) == np.argmax(y_true[test_nodes], axis=1)`).
- The two misclassified nodes in the plot are training nodes (`train_nodes`). In semi-supervised GCNs, training nodes contribute to the loss, but their predictions also depend on the graph structure (`A_hat`). Nodes with connections to both classes (e.g., boundary nodes in the karate club graph) may be misclassified if the model doesn't fully converge for all training nodes.
- The plot shows embeddings or predicted labels for all 34 nodes, highlighting two training nodes as misclassified.

Why This Happens

- **Semi-Supervised Learning**: The GNN uses `A_hat` to propagate information across the graph, so training node predictions are influenced by neighbors, not just their own labels. Some training nodes may have ambiguous connections, leading to misclassifications.
- **Model Capacity**: With `hidden_dim=[16, 2]` and two layers, the model may not perfectly fit all training nodes, especially those with complex graph connections.
- **Test Set Size**: The test set (10 nodes) is small, and random selection may have chosen "easier" nodes (e.g., those with clear connections to one class), leading to 100% accuracy, while training nodes include harder cases.

Recommended Actions

1. **Confirm Misclassified Nodes**:

```python
y_pred = model.forward_pass(A_hat, X)
misclassified = np.where(np.argmax(y_pred, axis=1) != np.argmax(y_true
```

```
print(f"Misclassified nodes: {misclassified}")
print(f"In training set? {[i in train_nodes for i in misclassified]}")
print(f"In test set? {[i in test_nodes for i in misclassified]}")
```

2. **Check Training and Test Accuracy**:

```
python

train_acc = np.mean(np.argmax(y_pred[train_nodes], axis=1) == np.argma
test_acc = np.mean(np.argmax(y_pred[test_nodes], axis=1) == np.argmax(
print(f"Training accuracy: {train_acc:.3f}")
print(f"Test accuracy: {test_acc:.3f}")
```

3.